Group name: _____ Group no.: _____

Polyglot

A compiler for mixed Python, C++ and Java-inspired syntax

Group members

01 Name: _____ Roll / ID: _____

02 Name: _____ Roll / ID: _____

03 Name: _____ Roll / ID: _____

04 Name: _____ Roll / ID: _____

Problem statement

Syntax can interrupt familiar logic

Students may know how to solve a problem but forget a language's exact syntax.

Programmers switching between languages can face the same friction.

A section of a C++ program may feel simpler to express using Python-style control flow.

We want to explore whether one compiler can accept supported spellings from several languages in the same program.

The basic idea

We plan to accept supported Python, C++ and Java-inspired syntax in one .poly file.

The compiler will recognize each construct automatically, normalize it and execute one program.

A proposed mixed program

One variable across three spellings

```
total = 2

int extra = 3;

total += extra

System.out.println(total);

std::cout << total << std::endl;

print("done")
```

Expected output

```
5

5

done
```

The planned compiler will resolve total and extra in one shared symbol table.

What we plan to build

An educational compiler for a deliberate subset

Supported syntax will mix between statements and inside nested blocks, with one set of semantic rules.

Variables and expressions

Integers, floats, Booleans and strings. Shared operators and checked assignments.

Functions

Parameters, return values, forward calls and recursion across syntax families.

Control flow

Conditionals, while loops and both for forms, including break and continue.

Output and diagnostics

print, cout and println. Clear errors with original source locations.

Outside scope: full language compatibility, classes, libraries and native linking

The planned compilation pipeline

Input: program.poly Build output: program.pbc

01	Lexer and layout	Tokens, indentation and braces
02	Recognition and parsing	Supported constructs and syntax evidence
03	Shared AST	Abstract syntax tree for all syntax forms
04	Semantic analysis	Bindings, types and initialization
05	Typed intermediate representation	Explicit operations and control flow
06	Bytecode and virtual machine	Reusable artifact and program execution

Automatic syntax recognition

Grammar and context will guide parsing

def, a colon and an indented suite will identify a Python-style construct.

Typed declarations and brace blocks will identify shared C++/Java-style forms.

cout and println will have explicit built-in grammar rules.

Some spellings belong to more than one family.

int count = 3;

We will record it as shared C++/Java syntax. Recognition will preserve its source span and the evidence used to parse it.

One shared meaning

Different spellings will use common rules

A common symbol table will let every supported syntax read and update the same bindings.

Primitive type aliases will normalize to common types. Assignments and function calls will receive static checks.

The analyzer will check scope, initialization and return paths. It will infer unannotated function types when constraints resolve consistently.

If a program is invalid, diagnostics will identify the source location and explain the error before execution.

Implementation tools

Python 3.11+ with a small dependency set

We will use typed data structures and separate modules for each compiler stage.

The command-line interface will support checking, running, building and inspecting programs.

Tokens, AST, IR and bytecode will remain inspectable for the demonstration.

Parsing

A handwritten lexer, recursive-descent parser and Pratt expression parser.

Execution

Custom bytecode and a stack-based VM. A versioned JSON artifact with a validated loader.

Verification

pytest, positive and negative fixtures, and comparisons across equivalent syntax forms.

Development approach

Tests will accompany each stage. Mixed programs will anchor acceptance.

01 Front end

Grammar and parsing

Define the subset, build layout handling and normalize mixed constructs into the AST.

02 Compiler and VM

Shared semantics

Add type checks, lower to typed IR and generate bytecode for the custom VM.

03 Validation

Examples and documentation

Test nested mixed programs, verify artifacts and prepare a reproducible demonstration.

Our final goal

A working compiler that builds and executes supported mixed programs.

An inspectable pipeline with clear diagnostics and reusable bytecode artifacts.

A documented demonstration showing shared behavior across syntax families.